

Polynomials Trajectory Planner

Preliminary Documentation

Gianluca Castellari

Summer 2026

Abstract

Symbology, nomenclature definitions. Theory of the polynomial profiles.

Contents

1	Symbols And Nomenclature	4
1.1	Intro And Scope	4
1.2	Guiding Conventions	4
1.3	Time Variables	4
1.4	Position And Trajectory In 1D	4
1.5	Derivatives In 1D	5
1.6	Boundary Conditions In 1D	5
1.7	State Notation	6
1.8	Position And Trajectory In 3D	6
1.9	Derivatives In 3D	6
1.10	Boundary Conditions In 3D	7
1.11	Segments, Multi-Segment Trajectories, And Blending	7
1.12	Constraint Notation	8
1.13	Reserved Orientation Notation	8
1.14	Procedure For Later Documents	8
1.15	Worked Symbolic Considerations	9
1.16	Open Questions	9
1.17	Related Documents	9
2	Ninth-Degree Polynomial Theory	9
2.1	Intro And Scope	9
2.2	Symbols Used	10
2.3	Why Degree Nine Is A Natural First Case	10
2.4	Canonical Form In The Time Domain	10
2.4.1	Reading The Canonical Form	11
2.5	Horner Form In The Time Domain	11
2.5.1	How Horner Form Is Built	11
2.6	Canonical Form Vs Horner Form	11
2.6.1	Canonical Form: Advantages	12
2.6.2	Canonical Form: Limitations	12
2.6.3	Horner Form: Advantages	12
2.6.4	Horner Form: Limitations	12
2.6.5	Practical Interpretation	12
2.7	General Derivative Rule	12
2.8	Derivatives Of The Ninth-Degree Polynomial	13
2.8.1	First Derivative: Velocity	13

2.8.2	Second Derivative: Acceleration	13
2.8.3	Third Derivative: Jerk	13
2.8.4	Fourth Derivative: Snap	13
2.8.5	Fifth Derivative: Crackle	13
2.8.6	Sixth Derivative: Pop	13
2.9	What Changes In 3D	14
2.10	Why Fewer Derivatives May Be Enough	14
2.10.1	Position And Velocity Only	14
2.10.2	Up To Acceleration	14
2.10.3	Up To Jerk	15
2.11	Why More Derivatives Can Be Useful	15
2.11.1	First Reason: Endpoint Specification Richness	15
2.11.2	Second Reason: Higher-Order Smoothness	15
2.11.3	Third Reason: Analytical Support For Later Constraints	15
2.12	Are Crackle And Pop Physically Necessary	15
2.13	The Special Role Of Snap In Degree Nine	15
2.14	A Practical Evaluation Procedure	16
2.15	Worked Symbolic Considerations	16
2.15.1	Example 1: Full Ninth-Degree Segment In 1D	16
2.15.2	Example 2: Why Higher Derivatives Matter For Extrema	16
2.15.3	Example 3: 3D Interpretation	16
2.16	Open Questions	17
2.17	Related Documents	17
3	Boundary Conditions	17
3.1	Intro And Scope	17
3.2	Symbols Used	17
3.3	What A Boundary Condition Is	18
3.4	Why The Number Of Boundary Conditions Depends On Degree	18
3.5	The Natural Symmetric Endpoint Pattern	19
3.6	Three Condition Classes	19
3.6.1	Necessary Conditions	19
3.6.2	Possible Conditions	19
3.6.3	Redundant Conditions	19
3.7	Condition Catalogue In 1D	20
3.8	Degree 3	20
3.8.1	Polynomial Structure	20
3.8.2	Necessary Conditions	20
3.8.3	Possible Conditions	21
3.8.4	Redundant Conditions	21
3.9	Degree 5	21
3.9.1	Polynomial Structure	21
3.9.2	Necessary Conditions	21
3.9.3	Possible Conditions	21
3.9.4	Redundant Conditions	22
3.10	Degree 7	22
3.10.1	Polynomial Structure	22
3.10.2	Necessary Conditions	22
3.10.3	Possible Conditions	22
3.10.4	Redundant Conditions	22
3.11	Degree 9	22
3.11.1	Polynomial Structure	22
3.11.2	Necessary Conditions	23
3.11.3	Possible Conditions	23

3.11.4	Redundant Conditions	23
3.12	Summary Table For 1D	23
3.13	Independence Matters As Much As Count	24
3.14	What Changes In 3D	24
3.15	Relation To The Data Model	24
3.16	Worked Symbolic Considerations	25
3.16.1	Example 1: Cubic Segment	25
3.16.2	Example 2: Quintic Segment	25
3.16.3	Example 3: Ninth-Degree Segment	25
3.17	Open Questions	26
3.18	Related Documents	26
4	Polynomial Coefficient Determination	26
4.1	Intro And Scope	26
4.2	Symbols Used	26
4.3	Problem Statement	27
4.4	Core Idea	27
4.5	Absolute-Time Formulation	27
4.5.1	General Polynomial	27
4.5.2	General Derivative Rule	28
4.5.3	Derivative Row Operators	28
4.6	Degree-Specific Linear Systems In Absolute Time	28
4.6.1	Degree 3	28
4.6.2	Degree 5	29
4.6.3	Degree 7	29
4.6.4	Degree 9	30
4.7	Step-By-Step Procedure In Absolute Time	30
4.8	Why Normalized Time Is Worth Introducing	30
4.9	Derivative Scaling Between t And τ	31
4.10	Degree-9 System In Normalized Time	31
4.11	How The Lower Degrees Fit The Same Normalized Pattern	33
4.11.1	Degree 3	33
4.11.2	Degree 5	33
4.11.3	Degree 7	33
4.11.4	Degree 9	33
4.12	Absolute Time Vs Normalized Time	33
4.12.1	Absolute Time: Main Advantages	33
4.12.2	Absolute Time: Main Limitations	34
4.12.3	Normalized Time: Main Advantages	34
4.12.4	Normalized Time: Main Limitations	34
4.12.5	Practical Interpretation For This Project	34
4.13	Recommended Determination Strategy	34
4.14	Independence And Solvability	34
4.15	Relation To The Data Model	35
4.16	What Changes In 3D	35
4.17	Worked Symbolic Considerations	36
4.17.1	Example 1: Degree 3	36
4.17.2	Example 2: Degree 9 In Normalized Time	36
4.17.3	Example 3: Why Normalized Time Helps In Multi-Segment Work	36
4.18	Open Questions	36
4.19	Related Documents	37

1 Symbols And Nomenclature

1.1 Intro And Scope

This document defines the notation used throughout the theory library and, later, in the software implementation notes. Its purpose is to remove ambiguity before entering derivations, algorithms, and design decisions.

The project works with polynomial trajectories in 1D first and then extends the same concepts to 3D when needed. The 2D case is intentionally omitted because it is considered a direct conceptual subset of the 3D case.

Orientation is included here only as reserved notation. Full theoretical treatment of orientation is explicitly postponed to a later phase.

1.2 Guiding Conventions

- Time-dependent quantities are written as functions of time, for example $x(t)$.
- Scalar quantities are written in plain italic notation, for example $x(t)$, $v(t)$, T .
- Three-dimensional vectors are written in bold, for example $\mathbf{p}(t)$.
- Initial and final values use subscripts 0 and f , for example x_0 , x_f , $\mathbf{p}_0$, $\mathbf{p}_f$.
- Segment indices use subscripts k , for example $x_k(t)$.
- Normalized time is written as τ , while absolute time is written as t .

1.3 Time Variables

The following symbols are used for time:

- t : absolute time variable.
- t_0 : initial time.
- t_f : final time.
- $T = t_f - t_0$: motion duration.
- τ : normalized time variable, usually defined as

$$\tau = \frac{t - t_0}{T}, \quad \tau \in [0, 1].$$

This distinction is useful because some derivations are clearer in absolute time, while others are more compact in normalized time.

1.4 Position And Trajectory In 1D

In one dimension, the trajectory is represented by a scalar position law:

$$x(t)$$

For a polynomial of degree n , the canonical form is:

$$x(t) = \sum_{i=0}^n a_i t^i$$

where:

- a_i are the polynomial coefficients;
- n is the polynomial degree.

When the project specifically refers to the ninth-degree polynomial, the notation becomes:

$$x(t) = a_0 + a_1t + a_2t^2 + \cdots + a_9t^9$$

The corresponding Horner form is written as:

$$x(t) = (((((((((a_9t + a_8)t + a_7)t + a_6)t + a_5)t + a_4)t + a_3)t + a_2)t + a_1)t + a_0)$$

This document only fixes the notation. The analytical comparison between canonical and Horner form belongs to `NinthDegreePolynomialTheory`.

1.5 Derivatives In 1D

The project adopts the following derivative chain:

- position: $x(t)$
- velocity: $\dot{x}(t)$
- acceleration: $\ddot{x}(t)$
- jerk: $x^{(3)}(t)$
- snap: $x^{(4)}(t)$
- crackle: $x^{(5)}(t)$
- pop: $x^{(6)}(t)$

For quick reading, the shorthand notation below is also accepted when useful:

$$v(t) = \dot{x}(t), \quad a(t) = \ddot{x}(t), \quad j(t) = x^{(3)}(t), \quad s(t) = x^{(4)}(t)$$

where:

- $v(t)$ denotes velocity;
- $a(t)$ denotes acceleration;
- $j(t)$ denotes jerk;
- $s(t)$ denotes snap.

For higher derivatives beyond snap, the explicit derivative notation $x^{(5)}(t)$ and $x^{(6)}(t)$ is preferred because it is less ambiguous than introducing too many extra letters.

1.6 Boundary Conditions In 1D

A boundary condition is a value imposed at the start or end of a segment. The standard notation is:

$$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$$

These symbols mean:

- x_0, x_f : initial and final position;
- v_0, v_f : initial and final velocity;
- a_0, a_f : initial and final acceleration;
- j_0, j_f : initial and final jerk;
- s_0, s_f : initial and final snap.

Not every polynomial degree requires the same set of boundary conditions. In later documents, boundary conditions will always be classified as:

- necessary;
- possible;

- redundant.

This classification depends on the polynomial degree under study.

1.7 State Notation

When it is useful to describe the motion condition at a single endpoint, the project uses the concept of state.

In 1D, a state can be written as:

$$\mathcal{S}_{1D} = (x, v, a, j, s)$$

or, if endpoint notation is needed:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

The symbol $\mathcal{S}$ (calligraphic) is used here as a compact state container. It is deliberately distinct, both visually and semantically, from s / $\mathbf{s}$, which remain reserved exclusively for snap (see *Derivatives In 1D*) — no disambiguation is required when both appear in the same context, since the two symbols no longer collide.

1.8 Position And Trajectory In 3D

In three dimensions, position is represented by a vector:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

The three scalar component laws are treated with the same notation already defined for 1D.

This means that a 3D polynomial trajectory can be understood as three coordinated scalar polynomial laws:

$$x(t), \quad y(t), \quad z(t)$$

or, compactly:

$$\mathbf{p}(t) = [x(t), y(t), z(t)]^T$$

1.9 Derivatives In 3D

The derivative hierarchy extends component-wise:

$$\dot{\mathbf{p}}(t), \quad \ddot{\mathbf{p}}(t), \quad \mathbf{p}^{(3)}(t), \quad \mathbf{p}^{(4)}(t)$$

The compact names are:

- velocity vector: $\mathbf{v}(t) = \dot{\mathbf{p}}(t)$
- acceleration vector: $\mathbf{a}(t) = \ddot{\mathbf{p}}(t)$
- jerk vector: $\mathbf{j}(t) = \mathbf{p}^{(3)}(t)$
- snap vector: $\mathbf{q}(t) = \mathbf{p}^{(4)}(t)$

The symbol $\mathbf{q}(t)$ is reserved here for vector snap only to avoid overloading the scalar letter s . If this creates confusion with quaternion notation in a future document, the explicit form $\mathbf{p}^{(4)}(t)$ should be preferred.

1.10 Boundary Conditions In 3D

The endpoint notation extends directly from the 1D case:

$$\mathbf{p}_0, \mathbf{p}_f, \mathbf{v}_0, \mathbf{v}_f, \mathbf{a}_0, \mathbf{a}_f, \mathbf{j}_0, \mathbf{j}_f, \mathbf{p}_0^{(4)}, \mathbf{p}_f^{(4)}$$

Each vector quantity contains three scalar components. For example:

$$\mathbf{v}_0 = \begin{bmatrix} v_{x,0} \\ v_{y,0} \\ v_{z,0} \end{bmatrix}$$

This reinforces the main project idea: 3D treatment is built from the 1D theory, then extended by vector composition.

1.11 Segments, Multi-Segment Trajectories, And Blending

For a single segment, the trajectory law may be written as:

$$x_k(t)$$

in 1D, or:

$$\mathbf{p}_k(t)$$

in 3D, where k is the segment index.

A complete multi-segment trajectory is represented as an ordered set:

$$\mathcal{T} = \{\text{segment}_1, \text{segment}_2, \dots, \text{segment}_N\}$$

When blending is introduced between consecutive segments, the project uses the following notation:

- $b_{k \rightarrow k+1}^{start}$: point where blending toward the next segment starts;
- $b_{k \rightarrow k+1}^{end}$: point where blending from the previous segment ends.

If the context is explicitly geometric, those symbols may be written as positions:

$$x_{b,k}^{start}, \quad x_{b,k}^{end}$$

in 1D, or:

$$\mathbf{p}_{b,k}^{start}, \quad \mathbf{p}_{b,k}^{end}$$

in 3D.

1.12 Constraint Notation

The project uses generic constraint symbols first, then specific ones when needed.

Generic notation:

$$C = \{\text{type}, \text{value}\}$$

Typical scalar limits in 1D are:

- v_{max} : maximum admissible velocity;
- a_{max} : maximum admissible acceleration;
- j_{max} : maximum admissible jerk;
- s_{max} : maximum admissible snap.

When a constant-velocity sub-segment is required, the notation is:

$$x_{cv}^{start}, \quad x_{cv}^{end}, \quad v_{cv}$$

in 1D, and:

$$\mathbf{p}_{cv}^{start}, \quad \mathbf{p}_{cv}^{end}, \quad v_{cv}$$

in 3D.

The scalar v_{cv} denotes the requested constant speed magnitude. If direction matters in 3D, the corresponding constant velocity vector should be written as:

$$\mathbf{v}_{cv}$$

1.13 Reserved Orientation Notation

Orientation is not developed here, but the following symbols are reserved so that future documents remain coherent:

- Euler angles: ϕ, θ, ψ
- rotation matrix: $\mathbf{R}$
- quaternion: $\mathbf{q}$ or q , depending on the chosen convention

Because $\mathbf{q}$ may also be used for snap in some literature, future orientation documents must explicitly declare the chosen convention before use. Until then:

- use $\mathbf{p}^{(4)}(t)$ when referring to vector snap in potentially ambiguous contexts;
- use $\mathbf{R}, \phi, \theta, \psi$ for reserved orientation notation;
- avoid quaternion derivations for now.

1.14 Procedure For Later Documents

To keep the full library consistent, every later document should follow this practical sequence:

1. start from the notation defined here;
2. introduce only the symbols actually needed by that topic;
3. keep the 1D derivation as the base case;
4. extend to 3D by component-wise interpretation or vector form;
5. declare any new symbol locally if it is not already defined here.

This simple procedure is important for study: it avoids re-learning a different notation in every document.

1.15 Worked Symbolic Considerations

The notation can already be read in a symbolic way without numerical data.

Example 1, 1D endpoint problem:

$$x(t) : (x_0, v_0, a_0, j_0, s_0) \rightarrow (x_f, v_f, a_f, j_f, s_f)$$

Example 2, 3D endpoint problem:

$$\mathbf{p}(t) : (\mathbf{p}_0, \mathbf{v}_0, \mathbf{a}_0) \rightarrow (\mathbf{p}_f, \mathbf{v}_f, \mathbf{a}_f)$$

Example 3, normalized-time formulation:

$$x(\tau) = \sum_{i=0}^n \alpha_i \tau^i, \quad \tau \in [0, 1]$$

where α_i are the coefficients expressed in normalized time.

These examples are intentionally symbolic because the project prioritizes structural understanding before numerical substitution.

1.16 Open Questions

- **Resolved in BoundaryConditions:** for each polynomial degree, the minimal boundary-condition set is established there (necessary / possible / redundant classification, with a summary table for degrees 3, 5, 7, 9).
- **Deferred to TrajectoryConstraints:** whether 3D constraints should be imposed component-wise or on vector magnitude. This question also recurs in **NinthDegreePolynomialTheory**; it is intentionally not decided here, in isolation, but once the max/min velocity/acceleration/jerk/snap analysis (Req. 7–10) is developed in that document, with enough technical grounding to decide properly.
- **Open — to resolve during future orientation work** (see **04-future-topics.md**): should quaternion notation use q or $\mathbf{q}$ once snap notation is already present? No urgency until orientation is actually developed.

1.17 Related Documents

- 06-theory-library-roadmap.md
- NinthDegreePolynomialTheory
- BoundaryConditions
- PolynomialCoefficientDetermination

2 Ninth-Degree Polynomial Theory

2.1 Intro And Scope

This document introduces the ninth-degree polynomial as the first complete reference case for the project. Its role is twofold:

- to define the polynomial in forms that are useful for derivation and implementation;
- to study the derivative chain from position up to the sixth derivative, so the later trajectory-planning documents can refer to a shared theoretical base.

The treatment starts in 1D, because that is the base case adopted by the project. The 3D case is then obtained by applying the same scalar law independently to the three Cartesian components.

This document focuses on Requirement 2 of `01-requirements.md`:

- canonical form and Horner form in the time domain;
- derivatives up to the sixth derivative;
- discussion of the usefulness of using more or fewer derivatives in trajectory planning.

2.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t : absolute time;
- t_0 : initial time;
- t_f : final time;
- $T = t_f - t_0$: motion duration;
- $x(t)$: scalar position law in 1D;
- $a_0, a_1, \dots, a_9$: polynomial coefficients;
- $\mathcal{S}_0, \mathcal{S}_f$: initial and final state containers when endpoint data are grouped compactly;
- $v(t), a(t), j(t)$: velocity, acceleration, and jerk;
- $x^{(4)}(t), x^{(5)}(t), x^{(6)}(t)$: snap, crackle, and pop.

When normalized time is briefly mentioned, the symbol τ is used with $\tau \in [0, 1]$.

2.3 Why Degree Nine Is A Natural First Case

A ninth-degree polynomial has ten coefficients. This makes it especially useful as a first full study case because a trajectory segment can be determined by ten scalar conditions if the formulation requires them.

In the standard symmetric endpoint formulation, those ten conditions are naturally obtained by prescribing:

$$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$$

where s denotes snap.

This does not mean that every problem must always be posed this way. It means that degree nine is the first degree in this project that naturally accommodates endpoint information up to snap at both ends of a segment.

Using the state notation introduced in `SymbolsAndNomenclature.md`, the same endpoint specification may also be grouped as:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

This compact notation is useful when the full endpoint information is treated as a single conceptual object rather than as a list of separate scalar conditions.

2.4 Canonical Form In The Time Domain

The canonical form is the direct polynomial expansion in powers of time:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 + a_6 t^6 + a_7 t^7 + a_8 t^8 + a_9 t^9$$

The same expression can be written compactly as:

$$x(t) = \sum_{i=0}^9 a_i t^i$$

This form is the most transparent one for theoretical work because every coefficient is visibly attached to its corresponding power of time.

2.4.1 Reading The Canonical Form

The canonical form is useful to read the structure of the motion law step by step:

1. the constant term a_0 is the value present even at $t = 0$;
2. the linear term $a_1 t$ contributes directly to initial slope;
3. higher powers progressively shape curvature and higher-order changes;
4. the highest-degree term $a_9 t^9$ gives the polynomial its ninth-degree character.

This reading is qualitative, but it is educationally useful because it connects algebraic structure to motion behavior.

2.5 Horner Form In The Time Domain

The same polynomial can be rewritten in nested form:

$$x(t) = a_0 + t \left(a_1 + t \left(a_2 + t \left(a_3 + t \left(a_4 + t \left(a_5 + t \left(a_6 + t \left(a_7 + t \left(a_8 + t a_9 \right) \right) \right) \right) \right) \right) \right) \right)$$

Equivalent right-nested notation is:

$$x(t) = (((((((((a_9 t + a_8)t + a_7)t + a_6)t + a_5)t + a_4)t + a_3)t + a_2)t + a_1)t + a_0)$$

These two expressions are algebraically identical. The difference is only in how the polynomial is organized for reading and evaluation.

2.5.1 How Horner Form Is Built

Starting from the canonical form:

$$x(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_9 t^9$$

factor t progressively:

$$x(t) = a_0 + t (a_1 + a_2 t + a_3 t^2 + \cdots + a_9 t^8)$$

then factor t again inside the parentheses:

$$x(t) = a_0 + t (a_1 + t (a_2 + a_3 t + a_4 t^2 + \cdots + a_9 t^7))$$

and continue until the fully nested structure is obtained.

This is not a new polynomial. It is the same polynomial rewritten so that each step uses one multiplication by t and one addition.

2.6 Canonical Form Vs Horner Form

The two forms serve different purposes.

2.6.1 Canonical Form: Advantages

- It shows the polynomial degree immediately.
- It makes symbolic differentiation straightforward.
- It is convenient when setting up coefficient-determination equations.
- It is easier to inspect manually during theoretical study.
- It exposes clearly which coefficient multiplies which power of time.

2.6.2 Canonical Form: Limitations

- It is less efficient for repeated numerical evaluation.
- A direct implementation may compute many powers of t explicitly.
- Large powers of time can make numerical scaling less convenient.
- It is less natural for recursive evaluation.

2.6.3 Horner Form: Advantages

- It is computationally efficient for numerical evaluation.
- It avoids explicit computation of powers such as t^7 , t^8 , and t^9 .
- It reduces the number of multiplications needed.
- It is often better suited for implementation in software.
- It is easy to evaluate repeatedly for many time samples.

2.6.4 Horner Form: Limitations

- It hides the power-by-power structure of the polynomial.
- It is less intuitive when deriving equations by hand.
- It is less transparent for teaching coefficient meaning.
- It is not the most convenient form for symbolic derivations.

2.6.5 Practical Interpretation

For this project, the best attitude is not to treat the two forms as competitors, but as complementary tools:

- canonical form is the preferred form for theory, derivation, and boundary-condition reasoning;
- Horner form is the preferred form for efficient numerical evaluation in software.

This is an important educational point: different mathematical forms of the same object become useful at different stages of the project.

2.7 General Derivative Rule

If

$$x(t) = \sum_{i=0}^9 a_i t^i$$

then the k -th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^9 \frac{i!}{(i-k)!} a_i t^{i-k}$$

This compact formula is useful because it shows the pattern behind all derivative expressions:

- every differentiation lowers the power of t by one;
- every differentiation multiplies the coefficient by the current exponent;

- terms of degree lower than k disappear in the k -th derivative.

2.8 Derivatives Of The Ninth-Degree Polynomial

Starting from

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 + a_6t^6 + a_7t^7 + a_8t^8 + a_9t^9$$

the derivatives up to the sixth are the following.

2.8.1 First Derivative: Velocity

$$v(t) = \dot{x}(t) = a_1 + 2a_2t + 3a_3t^2 + 4a_4t^3 + 5a_5t^4 + 6a_6t^5 + 7a_7t^6 + 8a_8t^7 + 9a_9t^8$$

Velocity is the first quantity that describes how position changes in time. It is also the most immediate kinematic limit in many planning problems.

2.8.2 Second Derivative: Acceleration

$$a(t) = \ddot{x}(t) = 2a_2 + 6a_3t + 12a_4t^2 + 20a_5t^3 + 30a_6t^4 + 42a_7t^5 + 56a_8t^6 + 72a_9t^7$$

Acceleration measures how velocity changes. It is central whenever smooth dynamic evolution is more important than position alone.

2.8.3 Third Derivative: Jerk

$$j(t) = x^{(3)}(t) = 6a_3 + 24a_4t + 60a_5t^2 + 120a_6t^3 + 210a_7t^4 + 336a_8t^5 + 504a_9t^6$$

Jerk is often introduced when acceleration continuity is not sufficient and sharper changes need to be controlled.

2.8.4 Fourth Derivative: Snap

$$x^{(4)}(t) = 24a_4 + 120a_5t + 360a_6t^2 + 840a_7t^3 + 1680a_8t^4 + 3024a_9t^5$$

Snap is especially relevant in a ninth-degree formulation because endpoint snap values can be included naturally among the ten scalar conditions.

2.8.5 Fifth Derivative: Crackle

$$x^{(5)}(t) = 120a_5 + 720a_6t + 2520a_7t^2 + 6720a_8t^3 + 15120a_9t^4$$

Crackle is less commonly used as a physical planning quantity, but it remains analytically useful because it is the derivative of snap.

2.8.6 Sixth Derivative: Pop

$$x^{(6)}(t) = 720a_6 + 5040a_7t + 20160a_8t^2 + 60480a_9t^3$$

Pop is the derivative of crackle. It is rarely a primary planning constraint, but it is part of the natural derivative chain of a ninth-degree polynomial and can help understand higher-order variation.

2.9 What Changes In 3D

The 3D case does not introduce a new scalar theory. It applies the same ninth-degree law to each Cartesian component:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

with

$$x(t) = \sum_{i=0}^9 a_i^{(x)} t^i, \quad y(t) = \sum_{i=0}^9 a_i^{(y)} t^i, \quad z(t) = \sum_{i=0}^9 a_i^{(z)} t^i$$

The same derivative chain applies component-wise:

$$\dot{\mathbf{p}}(t), \quad \ddot{\mathbf{p}}(t), \quad \mathbf{p}^{(3)}(t), \quad \mathbf{p}^{(4)}(t), \quad \mathbf{p}^{(5)}(t), \quad \mathbf{p}^{(6)}(t)$$

So, from the viewpoint of this document:

- the 1D theory is the real mathematical core;
- the 3D extension is obtained by repeating that theory on three components;
- extra difficulty in 3D comes later from coupling choices, shared duration, and constraint interpretation, not from a different polynomial basis.

2.10 Why Fewer Derivatives May Be Enough

In many trajectory-planning problems, using fewer derivatives is entirely reasonable.

2.10.1 Position And Velocity Only

If the problem only requires reaching a target position with prescribed endpoint velocities, a lower-order description may be sufficient. This is useful when:

- the motion is simple;
- dynamic smoothness beyond velocity is not critical;
- the polynomial degree can be kept lower.

The advantage is simplicity. The limitation is that acceleration and higher-order behavior are left less controlled.

2.10.2 Up To Acceleration

Using position, velocity, and acceleration is common when smoother motion is needed. This usually gives a better compromise between control and complexity.

Acceleration-level reasoning is already very meaningful because:

- many mechanical effects are more closely related to acceleration than to position alone;
- acceleration continuity usually improves motion quality substantially;
- it avoids some abrupt changes that remain hidden if only velocity is considered.

2.10.3 Up To Jerk

Including jerk is often valuable when the transition smoothness itself becomes a design concern. Even without discussing actuators in detail, jerk remains educationally important because it captures how aggressively acceleration changes.

For this project, jerk is especially useful because it forms the bridge between basic trajectory geometry and genuinely smooth motion planning.

2.11 Why More Derivatives Can Be Useful

Studying derivatives beyond jerk is not always necessary, but it can still be useful for at least three reasons.

2.11.1 First Reason: Endpoint Specification Richness

Degree nine allows endpoint conditions up to snap at both ends. If snap is part of the formulation, then studying snap explicitly is not optional; it belongs to the definition of the segment itself.

2.11.2 Second Reason: Higher-Order Smoothness

Even if crackle and pop are not imposed as boundary conditions, they still describe how higher-order quantities evolve. This is useful when the goal is not only to obtain a valid trajectory, but to understand how smooth it really is.

2.11.3 Third Reason: Analytical Support For Later Constraints

Higher derivatives appear naturally when later documents analyze extrema:

- extrema of velocity are related to acceleration;
- extrema of acceleration are related to jerk;
- extrema of jerk are related to snap;
- extrema of snap are related to crackle.

So even if crackle and pop are not used as physical constraints, they remain mathematically useful.

2.12 Are Crackle And Pop Physically Necessary

Usually, no.

For most practical planning formulations, crackle and pop are not primary quantities to prescribe directly. In this project they are mainly useful because:

- they complete the derivative hierarchy of the ninth-degree polynomial;
- they help interpret the behavior of snap and higher-order smoothness;
- they support later analytical reasoning about maxima, minima, and continuity.

In other words:

- position, velocity, acceleration, and jerk are often directly meaningful;
- snap may become directly meaningful in high-smoothness formulations;
- crackle and pop are often more useful as analytical companions than as primary design targets.

2.13 The Special Role Of Snap In Degree Nine

Among the higher derivatives, snap has a special status in this project.

The reason is structural: a ninth-degree polynomial has ten coefficients, and the symmetric endpoint data set

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

provides exactly ten scalar conditions.

This makes snap the highest derivative that fits naturally into the standard two-endpoint determination of a ninth-degree segment.

That is why this document studies derivatives beyond jerk, but still treats snap as the highest derivative with a particularly strong structural role.

2.14 A Practical Evaluation Procedure

When the polynomial must be evaluated numerically at many time instants, the following workflow is useful:

1. derive the equations in canonical form;
2. determine the coefficients from the chosen boundary conditions;
3. convert the polynomial to Horner form for repeated evaluation;
4. sample the motion over time to obtain position and derivative profiles.

This workflow matches the educational goal of the project:

- theory remains readable in canonical form;
- implementation remains efficient in Horner form.

2.15 Worked Symbolic Considerations

2.15.1 Example 1: Full Ninth-Degree Segment In 1D

Consider the symbolic segment

$$x(t) = \sum_{i=0}^9 a_i t^i$$

with endpoint data prescribed up to snap at both ends.

The key observation is not yet the solution for the coefficients, but the structural match:

- unknowns: $a_0, a_1, \dots, a_9$;
- scalar equations: initial and final values of position, velocity, acceleration, jerk, and snap.

This is why the ninth-degree polynomial is a natural theoretical anchor for the rest of the project.

2.15.2 Example 2: Why Higher Derivatives Matter For Extrema

Suppose a later document needs the maximum snap value. Then the stationary points of snap are found from:

$$\frac{d}{dt} \left(x^{(4)}(t) \right) = x^{(5)}(t) = 0$$

So crackle appears naturally even if it was never chosen as a planning constraint.

2.15.3 Example 3: 3D Interpretation

If

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

then a ninth-degree 3D segment is simply three synchronized ninth-degree scalar segments. The scalar theory does not change; only the bookkeeping grows.

2.16 Open Questions

- **Open — to resolve during the software design/implementation phase (src/)**: should derivative profiles be evaluated directly from canonical formulas or from Horner-style recursive schemes? This is not a theory-library decision — both forms are proven equivalent above; the choice depends on measured performance in the actual C#/WPF implementation, not on the underlying math.
- **Deferred to TrajectoryConstraints**: whether limits should be interpreted component-wise or on vector magnitude when extending from 1D to 3D. This duplicates the same open question in `SymbolsAndNomenclature` — see the consolidated note there; it will be resolved once developed in `TrajectoryConstraints`, not decided here.
- **Open — to resolve during the application/UI design phase (Req. 15)**: in the final application, which derivative levels should be visible by default in the charts, and which should remain optional?

2.17 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `BoundaryConditions`
- `PolynomialCoefficientDetermination`
- `TrajectoryConstraints`

3 Boundary Conditions

3.1 Intro And Scope

This document explains what boundary conditions are in the context of polynomial trajectory planning and lists the boundary conditions relevant to each polynomial degree currently in scope for the project: degree 3, 5, 7, and 9.

The goal of the document is educational before technical. It clarifies:

- what a boundary condition actually is;
- why the number of boundary conditions depends on the polynomial degree;
- how to distinguish necessary, possible, and redundant conditions;
- how the 1D reasoning extends to 3D.

This document covers the learning part of Requirement 3 in `01-requirements.md`.

3.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t_0 : initial time;
- t_f : final time;
- $T = t_f - t_0$: motion duration;
- $x(t)$: scalar position law in 1D;
- $\mathcal{S}_0, \mathcal{S}_f$: initial and final state containers when endpoint data are grouped compactly;
- x_0, x_f : initial and final position;
- v_0, v_f : initial and final velocity;
- a_0, a_f : initial and final acceleration;
- j_0, j_f : initial and final jerk;
- s_0, s_f : initial and final snap.

When needed, the generic k -th derivative is written as $x^{(k)}(t)$.

3.3 What A Boundary Condition Is

A boundary condition is a prescribed value imposed on the trajectory or on one of its derivatives at a specific time, usually at the beginning or at the end of a segment.

Typical examples are:

$$x(t_0) = x_0, \quad x(t_f) = x_f$$

for position, or:

$$\dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

for velocity.

So, in simple terms:

- a polynomial provides unknown coefficients;
- boundary conditions provide equations;
- when the number of independent equations matches the number of unknown coefficients, the polynomial can be determined uniquely.

This is the key idea behind the whole document.

3.4 Why The Number Of Boundary Conditions Depends On Degree

A polynomial of degree n has $n + 1$ coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + \cdots + a_nt^n$$

The unknowns are therefore:

$$a_0, a_1, a_2, \dots, a_n$$

which are $n + 1$ scalar unknowns.

If the duration T is already known and fixed, then the standard coefficient-determination problem requires:

$$n + 1$$

independent scalar boundary conditions.

This immediately gives the count for the degrees currently in scope:

- degree 3: 4 conditions;
- degree 5: 6 conditions;
- degree 7: 8 conditions;
- degree 9: 10 conditions.

3.5 The Natural Symmetric Endpoint Pattern

For the odd degrees used in this project, there is a natural symmetric pattern based on matching the same derivative orders at the start and at the end of the segment.

The pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This pattern is natural because it gives exactly the required number of scalar conditions:

$$2 \times \frac{n+1}{2} = n+1$$

for odd degree n .

This is not the only possible formulation, but it is the most direct and educational one for this project.

3.6 Three Condition Classes

To keep the terminology consistent across the project, boundary conditions are divided into three classes.

3.6.1 Necessary Conditions

Necessary conditions are the conditions that must be specified in the chosen formulation to determine the polynomial uniquely.

In this document, the default formulation is:

- fixed polynomial degree;
- fixed start and end times;
- endpoint conditions only;
- symmetric use of derivative orders at the two endpoints.

Under this formulation, the necessary conditions are the ones listed degree by degree in the next sections.

3.6.2 Possible Conditions

Possible conditions are conditions that can be used in broader or alternative formulations, but are not required in the default formulation.

Examples:

- replacing one endpoint derivative condition with another independent condition;
- prescribing an interior-point condition;
- using higher-order derivative data while freeing another parameter such as duration;
- using a non-symmetric formulation between start and end conditions.

So “possible” does not mean “needed here”. It means “admissible in some extended formulation”.

3.6.3 Redundant Conditions

Redundant conditions are extra independent conditions beyond what the fixed-degree, fixed-duration polynomial can satisfy uniquely.

If a degree- n polynomial with fixed duration already has $n+1$ independent scalar conditions, then adding another independent condition makes the problem overconstrained.

In that situation, one of the following happens:

- the system has no exact solution;
- one of the conditions is not actually independent;
- the formulation must be changed, for example by increasing degree or adding free parameters.

This is why the distinction between possible and redundant is important:

- possible means usable in some reformulated problem;
- redundant means extra within the current fixed formulation.

3.7 Condition Catalogue In 1D

Before listing the degrees one by one, it is useful to define the basic catalogue of endpoint conditions in 1D:

$$x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f, s_0, s_f$$

These correspond to:

- position at start and end;
- velocity at start and end;
- acceleration at start and end;
- jerk at start and end;
- snap at start and end.

For the degrees currently in scope, this catalogue is sufficient for the main endpoint-based theory.

When a full endpoint condition set is treated as a compact object, the state notation from `SymbolsAndNomenclature.md` may also be used. For example, in the degree-9 case:

$$\mathcal{S}_0 = (x_0, v_0, a_0, \dot{j}_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, \dot{j}_f, s_f)$$

This notation is particularly useful when discussing complete endpoint data sets without overloading the snap symbol s .

3.8 Degree 3

3.8.1 Polynomial Structure

A cubic polynomial has four coefficients:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

Therefore it needs four independent scalar conditions in the default formulation.

3.8.2 Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f$$

This means:

$$x(t_0) = x_0, \quad x(t_f) = x_f, \quad \dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

3.8.3 Possible Conditions

Possible conditions in broader formulations may include:

- acceleration values at one or both endpoints;
- jerk values at one or both endpoints;
- an interior position value;
- unknown duration combined with a different choice of endpoint conditions.

These conditions are possible only if the full formulation is changed so that the total number of independent unknowns and equations remains balanced.

3.8.4 Redundant Conditions

In the default cubic formulation with fixed duration, the following types of additions are redundant:

- prescribing a_0 or a_f in addition to x_0, x_f, v_0, v_f ;
- prescribing j_0 or j_f in addition to the same set;
- adding any extra independent endpoint condition beyond the original four.

So the basic educational message is simple: a cubic segment naturally controls position and velocity, not the full higher-order hierarchy.

3.9 Degree 5

3.9.1 Polynomial Structure

A quintic polynomial has six coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

Therefore it needs six independent scalar conditions in the default formulation.

3.9.2 Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f$$

This means:

$$x(t_0) = x_0, \quad x(t_f) = x_f$$

$$\dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

$$\ddot{x}(t_0) = a_0, \quad \ddot{x}(t_f) = a_f$$

3.9.3 Possible Conditions

Possible conditions in extended formulations may include:

- jerk at one or both endpoints;
- one or more interior conditions;
- duration treated as an unknown;
- a non-symmetric distribution of endpoint derivative conditions.

3.9.4 Redundant Conditions

In the default quintic formulation with fixed duration, the following become redundant if added independently:

- j_0, j_f ;
- any snap endpoint condition;
- any extra independent condition beyond the six already listed.

The quintic therefore extends the cubic case by naturally controlling acceleration at the boundaries.

3.10 Degree 7

3.10.1 Polynomial Structure

A seventh-degree polynomial has eight coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 + a_6t^6 + a_7t^7$$

Therefore it needs eight independent scalar conditions in the default formulation.

3.10.2 Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f$$

This means that position, velocity, acceleration, and jerk are prescribed at both endpoints.

3.10.3 Possible Conditions

Possible conditions in broader formulations may include:

- snap at one or both endpoints;
- interior constraints;
- alternative mixes of endpoint data;
- duration or other parameters promoted to unknowns.

3.10.4 Redundant Conditions

In the default seventh-degree formulation with fixed duration, the following become redundant if added independently:

- s_0, s_f ;
- any extra endpoint condition beyond the eight already used;
- any other independent scalar condition that does not replace an existing one.

The educational interpretation is that the seventh-degree polynomial is the first degree in this project that naturally includes jerk at both ends.

3.11 Degree 9

3.11.1 Polynomial Structure

A ninth-degree polynomial has ten coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 + a_6t^6 + a_7t^7 + a_8t^8 + a_9t^9$$

Therefore it needs ten independent scalar conditions in the default formulation.

3.11.2 Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$$

This means that position, velocity, acceleration, jerk, and snap are prescribed at both endpoints.

3.11.3 Possible Conditions

Possible conditions in broader formulations may include:

- crackle-related information if the problem is reformulated;
- interior-point constraints;
- asymmetric endpoint specifications;
- duration included among the unknowns;
- optimization-based or approximate fitting formulations.

3.11.4 Redundant Conditions

In the default ninth-degree formulation with fixed duration, the following become redundant if added independently:

- any additional crackle or pop endpoint condition;
- any extra interior scalar condition that is imposed without releasing another condition or adding unknowns;
- any extra independent scalar condition beyond the ten already used.

This is why snap has a special role in the degree-nine case: it is the highest derivative that still fits naturally into the standard symmetric endpoint formulation.

3.12 Summary Table For 1D

The degree-by-degree classification can be summarized as follows.

Degree	Coefficients	Necessary conditions in the default formulation	Examples of possible extra conditions in broader formulations	Redundant in the default fixed formulation
3	4	x_0, x_f, v_0, v_f	a_0, a_f , interior position, unknown duration	any fifth independent condition
5	6	$x_0, x_f, v_0, v_f, a_0, a_f$	j_0, j_f , interior conditions, unknown duration	any seventh independent condition
7	8	$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f$	s_0, s_f , interior conditions, unknown duration	any ninth independent condition
9	10	$x_0, x_f, v_0, v_f, a_0, a_f, j_0, j_f, s_0, s_f$	crackle-related data, interior conditions, unknown duration	any eleventh independent condition

This table should be read carefully:

- the “necessary” column refers only to the standard formulation adopted as default in this project;
- the “possible” column does not mean those conditions are added on top unchanged;
- the “redundant” column means “redundant unless the formulation itself is changed”.

3.13 Independence Matters As Much As Count

Counting conditions is necessary, but not sufficient. The conditions must also be independent.

For example, a degree-5 polynomial needs six scalar conditions, but not every group of six conditions is equally well chosen. If two conditions are linearly dependent in the coefficient system, they do not provide six independent equations.

So the complete rule is:

- correct number of conditions;
- independence of those conditions;
- consistency with the chosen formulation.

This point will become especially important in `PolynomialCoefficientDetermination`.

3.14 What Changes In 3D

The logic does not change in 3D. What changes is that the scalar 1D problem is applied to three Cartesian components.

If the three coordinates are treated independently, then each axis has its own set of scalar boundary conditions:

$$x_0, x_f, v_{x,0}, v_{x,f}, \dots$$

$$y_0, y_f, v_{y,0}, v_{y,f}, \dots$$

$$z_0, z_f, v_{z,0}, v_{z,f}, \dots$$

In compact vector notation, the corresponding endpoint conditions are:

$$\mathbf{p}_0, \mathbf{p}_f, \mathbf{v}_0, \mathbf{v}_f, \mathbf{a}_0, \mathbf{a}_f, \mathbf{j}_0, \mathbf{j}_f, \mathbf{p}_0^{(4)}, \mathbf{p}_f^{(4)}$$

The educational idea is still the same:

- first understand the scalar case;
- then apply the same rule component-wise in 3D.

The main difference in 3D is therefore not the nature of the boundary conditions, but the bookkeeping and the later interpretation of multi-axis coordination.

3.15 Relation To The Data Model

`02-data-model.md` now defines `BoundaryConditions` as:

$$\text{BoundaryConditions} = \{\text{necessaryConditions}[], \text{possibleConditions}[], \text{redundantConditions}[]\}$$

This is the degree-parametrized, classified structure that this document’s necessary/possible/redundant classification calls for — the revision has been carried out, replacing the earlier fixed set of ten scalar fields (`startPosition/endPosition`, etc.) that matched only the degree-9 default formulation.

The three arrays hold:

- `necessaryConditions[]`: the conditions required by the chosen degree and formulation;
- `possibleConditions[]`: conditions usable in alternative formulations;
- `redundantConditions[]`: conditions that exceed the default fixed formulation.

What remains open is not *whether* to revise the structure (done), but the internal representation of each individual condition within those arrays — see the corresponding open question below, to be resolved together with `PolynomialCoefficientDetermination`.

In later implementation, each condition will likely need a more explicit internal representation, for example:

- derivative order;
- endpoint identifier;
- symbolic or numeric value.

3.16 Worked Symbolic Considerations

3.16.1 Example 1: Cubic Segment

For a cubic segment with fixed duration, the unknown coefficients are:

$$a_0, a_1, a_2, a_3$$

The natural endpoint conditions are:

$$x_0, x_f, v_0, v_f$$

This gives four independent scalar equations for four unknowns.

If one now adds a_0 and a_f as prescribed endpoint accelerations without changing the formulation, the cubic problem becomes overconstrained.

3.16.2 Example 2: Quintic Segment

For a quintic segment, the unknown coefficients are:

$$a_0, a_1, a_2, a_3, a_4, a_5$$

The natural endpoint conditions are:

$$x_0, x_f, v_0, v_f, a_0, a_f$$

This is why quintic polynomials are often associated with endpoint control up to acceleration.

3.16.3 Example 3: Ninth-Degree Segment

For a ninth-degree segment, the natural symmetric endpoint set is:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

This gives ten scalar conditions, matching the ten coefficients of the polynomial exactly.

That exact structural match is one of the main reasons why the degree-nine case is the central reference of the project.

3.17 Open Questions

- **Resolved:** whether 02-data-model.md's `BoundaryConditions` structure should be revised to the degree-parametrized, classified form — it has been, see *Relation To The Data Model*. **Still open, deferred to PolynomialCoefficientDetermination:** the internal representation of each individual condition within `necessaryConditions[]` / `possibleConditions[]` / `redundantConditions[]` (by derivative order vs. a more descriptive enum-based type) — this is `PolynomialCoefficientDetermination`'s own Open Question on the same topic, not yet decided there either.
- **Open — to resolve if/when alternative formulations are introduced** (interior points, optimization-based fitting): should the project keep the same necessary/possible/redundant naming, or add a fourth category for “active in the current variant”? No urgency yet, since the project still uses the fixed default formulation.
- **Open — to resolve during the application/UI design phase** (Req. 15): for 3D trajectories, should the user interface expose boundary conditions axis by axis, or primarily in vector form?

3.18 Related Documents

- 06-theory-library-roadmap.md
- SymbolsAndNomenclature.md
- NinthDegreePolynomialTheory.md
- PolynomialCoefficientDetermination
- 02-data-model.md

4 Polynomial Coefficient Determination

4.1 Intro And Scope

This document explains how to determine the coefficients of a polynomial trajectory once the necessary boundary data has been specified.

It covers two connected themes:

- the technical procedure required by Requirement 3 in 01-requirements.md;
- the learning-oriented comparison between absolute time and normalized time required by Requirement 4.

The discussion starts in 1D, because that is the conceptual base of the project. The 3D case is then obtained by applying the same scalar procedure to the Cartesian components.

The focus is on degrees 3, 5, 7, and 9, which are the degrees currently in scope for the project.

4.2 Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t : absolute time;
- t_0 : initial time;
- t_f : final time;
- $T = t_f - t_0$: segment duration;
- $\tau = \frac{t-t_0}{T}$: normalized time;
- $x(t)$: scalar position law in 1D written in absolute time;
- $x(\tau)$: scalar position law in normalized time;
- n : polynomial degree;
- a_i : coefficients of the polynomial when written in absolute time;
- α_i : coefficients of the polynomial when written in normalized time;
- S_0, S_f : endpoint state containers when compact notation is useful.

When derivatives are written in the normalized variable, the notation

$$\frac{d^k x}{d\tau^k}$$

is preferred in order to keep it clearly distinct from derivatives with respect to absolute time.

4.3 Problem Statement

The coefficient-determination problem can be stated as follows.

Given:

- a polynomial degree n ;
- a fixed segment duration T ;
- the necessary boundary conditions for that degree;

determine the polynomial coefficients so that all required endpoint conditions are satisfied exactly.

For the degrees currently used in the project, the default endpoint pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This means that coefficient determination is always a square linear problem in the default formulation:

- degree 3: 4 unknown coefficients, 4 scalar equations;
- degree 5: 6 unknown coefficients, 6 scalar equations;
- degree 7: 8 unknown coefficients, 8 scalar equations;
- degree 9: 10 unknown coefficients, 10 scalar equations.

4.4 Core Idea

A polynomial trajectory has unknown coefficients. Each boundary condition produces one scalar equation in those coefficients.

So the practical workflow is always:

1. choose the polynomial degree;
2. write the polynomial and its derivatives;
3. evaluate the required derivatives at the endpoints;
4. assemble the linear system;
5. solve for the coefficients;
6. verify the result by substitution.

The mathematical heart of the method is therefore not a special trick, but a structured linear system.

4.5 Absolute-Time Formulation

4.5.1 General Polynomial

In absolute time, the polynomial is written as:

$$x(t) = \sum_{i=0}^n a_i t^i$$

with unknown coefficient vector:

$$\mathbf{a} = [a_0 \quad a_1 \quad \dots \quad a_n]^T$$

4.5.2 General Derivative Rule

The k -th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^n \frac{i!}{(i-k)!} a_i t^{i-k}$$

This formula is enough to generate every row of the coefficient-determination system.

4.5.3 Derivative Row Operators

For compact notation, define the row vector $\mathbf{D}_k(t)$ as the coefficient row that maps $\mathbf{a}$ to the k -th derivative value at time t :

$$x^{(k)}(t) = \mathbf{D}_k(t) \mathbf{a}$$

For degree 9, the first five rows are:

$$\mathbf{D}_0(t) = [1 \quad t \quad t^2 \quad t^3 \quad t^4 \quad t^5 \quad t^6 \quad t^7 \quad t^8 \quad t^9]$$

$$\mathbf{D}_1(t) = [0 \quad 1 \quad 2t \quad 3t^2 \quad 4t^3 \quad 5t^4 \quad 6t^5 \quad 7t^6 \quad 8t^7 \quad 9t^8]$$

$$\mathbf{D}_2(t) = [0 \quad 0 \quad 2 \quad 6t \quad 12t^2 \quad 20t^3 \quad 30t^4 \quad 42t^5 \quad 56t^6 \quad 72t^7]$$

$$\mathbf{D}_3(t) = [0 \quad 0 \quad 0 \quad 6 \quad 24t \quad 60t^2 \quad 120t^3 \quad 210t^4 \quad 336t^5 \quad 504t^6]$$

$$\mathbf{D}_4(t) = [0 \quad 0 \quad 0 \quad 0 \quad 24 \quad 120t \quad 360t^2 \quad 840t^3 \quad 1680t^4 \quad 3024t^5]$$

These rows are directly derived from `NinthDegreePolynomialTheory.md`.

4.6 Degree-Specific Linear Systems In Absolute Time

The default square systems for the degrees in scope are obtained by stacking the derivative rows at t_0 and t_f .

4.6.1 Degree 3

For

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

use:

$$\mathbf{a}_3 = [a_0 \quad a_1 \quad a_2 \quad a_3]^T$$

and:

$$\mathbf{b}_3 = [x_0 \quad v_0 \quad x_f \quad v_f]^T$$

The system is:

$$\mathbf{A}_3^{abs} \mathbf{a}_3 = \mathbf{b}_3$$

with:

$$\mathbf{A}_3^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \end{bmatrix}$$

where each row is truncated to degree 3.

4.6.2 Degree 5

Use:

$$\mathbf{b}_5 = [x_0 \quad v_0 \quad a_0 \quad x_f \quad v_f \quad a_f]^T$$

and:

$$\mathbf{A}_5^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \end{bmatrix}$$

again truncated to degree 5.

4.6.3 Degree 7

Use:

$$\mathbf{b}_7 = [x_0 \quad v_0 \quad a_0 \quad j_0 \quad x_f \quad v_f \quad a_f \quad j_f]^T$$

and:

$$\mathbf{A}_7^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \end{bmatrix}$$

truncated to degree 7.

4.6.4 Degree 9

Use the compact state notation:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

or, equivalently, the endpoint vector:

$$\mathbf{b}_9 = [x_0 \quad v_0 \quad a_0 \quad j_0 \quad s_0 \quad x_f \quad v_f \quad a_f \quad j_f \quad s_f]^T$$

Then:

$$\mathbf{A}_9^{abs} \mathbf{a}_9 = \mathbf{b}_9$$

with:

$$\mathbf{A}_9^{abs} = \begin{bmatrix} \mathbf{D}_0(t_0) \\ \mathbf{D}_1(t_0) \\ \mathbf{D}_2(t_0) \\ \mathbf{D}_3(t_0) \\ \mathbf{D}_4(t_0) \\ \mathbf{D}_0(t_f) \\ \mathbf{D}_1(t_f) \\ \mathbf{D}_2(t_f) \\ \mathbf{D}_3(t_f) \\ \mathbf{D}_4(t_f) \end{bmatrix}$$

This is the direct absolute-time coefficient-determination system for the degree-9 case.

4.7 Step-By-Step Procedure In Absolute Time

The practical procedure in absolute time is the same for every supported degree.

1. Choose the degree n and identify the necessary endpoint conditions from `BoundaryConditions.md`.
2. Write the polynomial $x(t)$ in canonical form with unknown coefficients.
3. Compute the derivatives needed by that degree.
4. Evaluate those derivatives at t_0 and t_f .
5. Assemble the matrix $\mathbf{A}^{abs}$ and the right-hand side vector $\mathbf{b}$.
6. Solve the linear system for the coefficient vector $\mathbf{a}$.
7. Substitute the solution back into the polynomial and verify all endpoint conditions.

Conceptually, this is simple. In practice, the algebra becomes longer as the degree increases.

4.8 Why Normalized Time Is Worth Introducing

Absolute time works directly, but it is not always the most convenient variable.

When the segment duration is fixed, it is often useful to define:

$$\tau = \frac{t - t_0}{T}, \quad \tau \in [0, 1]$$

and write the same segment as:

$$x(\tau) = \sum_{i=0}^n \alpha_i \tau^i$$

The important idea is that:

- t belongs to the physical time axis;
- τ belongs to a normalized local segment axis;
- the two descriptions represent the same trajectory segment, but with different coefficients.

If $t_0 \neq 0$, the sets $\{a_i\}$ and $\{\alpha_i\}$ are not related by a simple one-to-one scaling. The normalized representation is centered on the local interval $[0, 1]$, whereas the absolute-time representation is written directly in powers of the global variable t .

4.9 Derivative Scaling Between t And τ

Because

$$\tau = \frac{t - t_0}{T}$$

we have:

$$\frac{d\tau}{dt} = \frac{1}{T}$$

Therefore:

$$\frac{dx}{dt} = \frac{dx}{d\tau} \frac{d\tau}{dt} = \frac{1}{T} \frac{dx}{d\tau}$$

and, more generally:

$$\frac{d^k x}{dt^k} = \frac{1}{T^k} \frac{d^k x}{d\tau^k}$$

Equivalently:

$$\frac{d^k x}{d\tau^k} = T^k \frac{d^k x}{dt^k}$$

This scaling rule is the key bridge between physical endpoint data and the normalized-time coefficient system.

4.10 Degree-9 System In Normalized Time

For degree 9, write:

$$x(\tau) = \alpha_0 + \alpha_1 \tau + \alpha_2 \tau^2 + \alpha_3 \tau^3 + \alpha_4 \tau^4 + \alpha_5 \tau^5 + \alpha_6 \tau^6 + \alpha_7 \tau^7 + \alpha_8 \tau^8 + \alpha_9 \tau^9$$

with coefficient vector:

$$\boldsymbol{\alpha} = [\alpha_0 \quad \alpha_1 \quad \alpha_2 \quad \alpha_3 \quad \alpha_4 \quad \alpha_5 \quad \alpha_6 \quad \alpha_7 \quad \alpha_8 \quad \alpha_9]^T$$

At $\tau = 0$, the first five endpoint equations become:

$$x(0) = \alpha_0 = x_0$$

$$\frac{dx}{d\tau}(0) = \alpha_1 = Tv_0$$

$$\frac{d^2x}{d\tau^2}(0) = 2\alpha_2 = T^2a_0$$

$$\frac{d^3x}{d\tau^3}(0) = 6\alpha_3 = T^3j_0$$

$$\frac{d^4x}{d\tau^4}(0) = 24\alpha_4 = T^4s_0$$

At $\tau = 1$, the remaining equations are:

$$x(1) = \sum_{i=0}^9 \alpha_i = x_f$$

$$\frac{dx}{d\tau}(1) = \alpha_1 + 2\alpha_2 + 3\alpha_3 + 4\alpha_4 + 5\alpha_5 + 6\alpha_6 + 7\alpha_7 + 8\alpha_8 + 9\alpha_9 = Tv_f$$

$$\frac{d^2x}{d\tau^2}(1) = 2\alpha_2 + 6\alpha_3 + 12\alpha_4 + 20\alpha_5 + 30\alpha_6 + 42\alpha_7 + 56\alpha_8 + 72\alpha_9 = T^2a_f$$

$$\frac{d^3x}{d\tau^3}(1) = 6\alpha_3 + 24\alpha_4 + 60\alpha_5 + 120\alpha_6 + 210\alpha_7 + 336\alpha_8 + 504\alpha_9 = T^3j_f$$

$$\frac{d^4x}{d\tau^4}(1) = 24\alpha_4 + 120\alpha_5 + 360\alpha_6 + 840\alpha_7 + 1680\alpha_8 + 3024\alpha_9 = T^4s_f$$

So the normalized system is:

$$\mathbf{A}_9^{norm} \boldsymbol{\alpha} = \mathbf{b}_9^{norm}$$

with:

$$\mathbf{A}_9^{norm} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 24 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 0 & 0 & 2 & 6 & 12 & 20 & 30 & 42 & 56 & 72 \\ 0 & 0 & 0 & 6 & 24 & 60 & 120 & 210 & 336 & 504 \\ 0 & 0 & 0 & 0 & 24 & 120 & 360 & 840 & 1680 & 3024 \end{bmatrix}$$

and:

$$\mathbf{b}_9^{norm} = [x_0 \quad Tv_0 \quad T^2a_0 \quad T^3j_0 \quad T^4s_0 \quad x_f \quad Tv_f \quad T^2a_f \quad T^3j_f \quad T^4s_f]^T$$

This is one of the main practical advantages of normalized time: for a fixed degree, the matrix is constant and can be reused for every segment.

4.11 How The Lower Degrees Fit The Same Normalized Pattern

The same normalized-time logic applies to every supported degree.

4.11.1 Degree 3

Use:

$$\mathbf{b}_3^{norm} = [x_0 \quad Tv_0 \quad x_f \quad Tv_f]^T$$

with a constant 4×4 normalized matrix.

4.11.2 Degree 5

Use:

$$\mathbf{b}_5^{norm} = [x_0 \quad Tv_0 \quad T^2a_0 \quad x_f \quad Tv_f \quad T^2a_f]^T$$

with a constant 6×6 normalized matrix.

4.11.3 Degree 7

Use:

$$\mathbf{b}_7^{norm} = [x_0 \quad Tv_0 \quad T^2a_0 \quad T^3j_0 \quad x_f \quad Tv_f \quad T^2a_f \quad T^3j_f]^T$$

with a constant 8×8 normalized matrix.

4.11.4 Degree 9

Use the degree-9 system written in the previous section.

So the pattern is stable:

- the normalized matrix depends only on degree;
- the right-hand side contains the physical endpoint data scaled by powers of T .

4.12 Absolute Time Vs Normalized Time

The two formulations represent the same trajectory, but they are useful in different ways.

4.12.1 Absolute Time: Main Advantages

- It keeps the polynomial directly expressed in the physical variable t .
- It makes the final law immediately readable in the original time axis.
- It avoids introducing an extra variable.
- It is conceptually straightforward when $t_0 = 0$ and durations are simple.

4.12.2 Absolute Time: Main Limitations

- The system matrix changes with t_0 and t_f .
- Large or awkward time values generate large powers such as t_f^8 or t_f^9 .
- The algebra becomes less uniform from one segment to another.
- Numerical conditioning can become worse when absolute times are large.

4.12.3 Normalized Time: Main Advantages

- The segment always lives on the same interval $[0, 1]$.
- For a given degree, the system matrix is constant and reusable.
- The formulation is usually cleaner for symbolic derivation.
- Numerical scaling is often better because the variable does not grow beyond the local interval.
- Multi-segment implementations become more uniform because every segment uses the same local coordinate.

4.12.4 Normalized Time: Main Limitations

- Physical derivatives must be rescaled by powers of T .
- The coefficients α_i are not the same as the absolute-time coefficients a_i .
- If a final expression explicitly in powers of t is required, an extra conversion step is needed.
- The method introduces one extra conceptual layer for beginners.

4.12.5 Practical Interpretation For This Project

For this project, the most balanced approach is:

- use normalized time as the main working variable for coefficient determination;
- keep absolute time as the interpretation layer in which physical durations and derivative values are specified;
- evaluate the final trajectory numerically either in normalized time or, if needed, after conversion to absolute time.

This keeps the derivation clean without losing contact with physical meaning.

4.13 Recommended Determination Strategy

The following strategy is recommended for the project.

1. Express the trajectory segment in normalized time.
2. Build the constant normalized matrix associated with the chosen degree.
3. Assemble the right-hand side from the physical endpoint data scaled by T^k .
4. Solve for the normalized coefficients α_i .
5. Evaluate the trajectory and its derivatives using τ as the local segment variable.
6. Convert to an absolute-time polynomial only if that representation is explicitly needed.

This recommendation is especially useful for software implementation because it separates:

- physical inputs;
- degree-dependent matrix structure;
- reusable numerical evaluation.

4.14 Independence And Solvability

Counting equations is not enough. The boundary conditions must also be independent, and the matrix must be invertible.

So a well-posed coefficient-determination problem requires:

- the correct number of scalar conditions for the chosen degree;
- independence of those conditions;
- a consistent time definition with $T \neq 0$.

If these conditions fail, the system may become:

- underdetermined, if too few independent conditions are supplied;
- overdetermined, if too many independent conditions are imposed;
- singular, if the conditions are formally counted correctly but not independent.

This connects directly to the necessary / possible / redundant classification introduced in `BoundaryConditions.md`.

4.15 Relation To The Data Model

`02-data-model.md` currently defines:

$$\text{Polynomial} = \{\text{degree}, \text{coefficients}, \text{boundaryConditions}\}$$

and:

$$\text{BoundaryConditions} = \{\text{necessaryConditions}[], \text{possibleConditions}[], \text{redundantConditions}[]\}$$

From the viewpoint of coefficient determination, this means:

- **degree** selects the size and structure of the determination system;
- **necessaryConditions[]** provides the data that actually feeds the square linear system;
- **possibleConditions[]** may become relevant only in alternative formulations;
- **redundantConditions[]** should not be injected into the default fixed-degree system unchanged.

So the coefficient-determination procedure is the first place where the theoretical classification of boundary conditions becomes operational.

A future implementation-oriented refinement will likely need each necessary condition to carry at least:

- derivative order;
- endpoint identifier;
- scalar or vector value;
- dimensional context (1D or 3D).

4.16 What Changes In 3D

The determination logic does not change in 3D. The same scalar problem is solved component-wise.

For example, in a degree-9 segment:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

the project can determine:

- one coefficient vector for the x component;
- one coefficient vector for the y component;
- one coefficient vector for the z component.

If the duration T is shared by all three axes, then the same normalized matrix is reused three times with three different right-hand side vectors.

This is one of the strongest reasons for keeping the 1D theory as the base case: the 3D extension is structurally repetitive rather than conceptually new.

4.17 Worked Symbolic Considerations

4.17.1 Example 1: Degree 3

For a cubic segment with fixed duration, the unknown coefficients are:

$$a_0, a_1, a_2, a_3$$

The natural endpoint data are:

$$x_0, v_0, x_f, v_f$$

These four scalar conditions generate a 4×4 square system. If the system matrix is invertible, the four coefficients are determined uniquely.

4.17.2 Example 2: Degree 9 In Normalized Time

For the degree-9 case, the endpoint data may be grouped as:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

The normalized system then uses:

$$x_0, T v_0, T^2 a_0, T^3 j_0, T^4 s_0, x_f, T v_f, T^2 a_f, T^3 j_f, T^4 s_f$$

as right-hand side data. The matrix stays constant; only the scaled endpoint vector changes from segment to segment.

4.17.3 Example 3: Why Normalized Time Helps In Multi-Segment Work

Suppose two trajectory segments have the same polynomial degree but different durations. In absolute time, their matrices are different because their endpoint times are different.

In normalized time, both segments use the same coefficient matrix. Only the right-hand side changes through the powers of each segment duration:

- one segment uses T_1, T_1^2, T_1^3 , and so on;
- the other uses T_2, T_2^2, T_2^3 , and so on.

This is mathematically cleaner and implementation-friendly.

4.18 Open Questions

- **Open — to resolve during software implementation (src/):** should the application store only normalized coefficients internally, or also keep an absolute-time coefficient expansion for inspection/export?
- **Deferred to the later constraints/blending documents:** when a formulation moves beyond the default necessary-condition set, should the software solve a restructured exact system, or transition to constrained/optimization-based formulations?

- **Open — to resolve during API/data-structure design:** should each condition within `necessaryConditions[]` / `possibleConditions[]` / `redundantConditions[]` be stored as a generic condition record, or as degree-specific strongly typed structures for degrees 3, 5, 7, and 9? (The classified array structure itself is already in place in `02-data-model.md`; only the internal representation of each entry remains open. Same open question as in `BoundaryConditions.md` — not yet decided in either document.)

4.19 Related Documents

- `06-theory-library-roadmap.md`
- `SymbolsAndNomenclature.md`
- `NinthDegreePolynomialTheory.md`
- `BoundaryConditions.md`
- `TrajectoryConstraints`
- `02-data-model.md`